

5주차

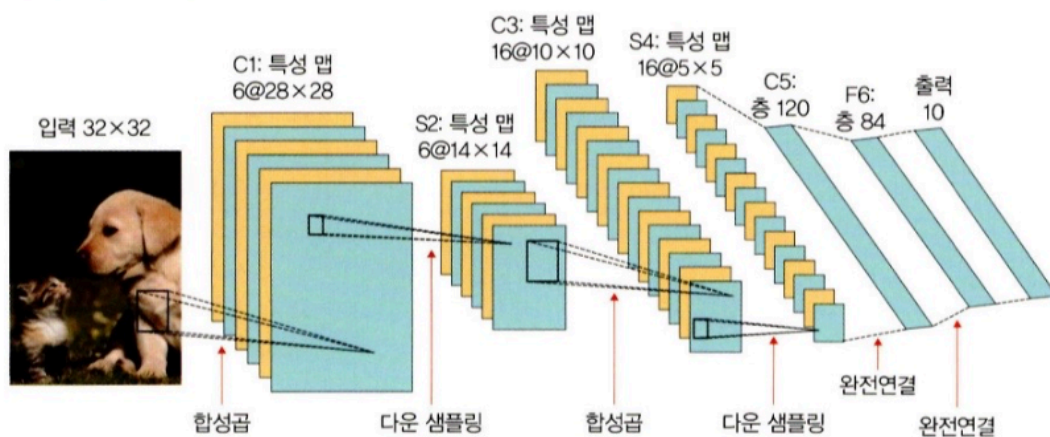
6장 합성곱 신경망 II

6.1 이미지 분류를 위한 신경망 개념 정리

이미지 분류 (Image Classification)

- 입력 이미지 안에 특정 대상이 존재하는지 판단하는 문제.
- 예: 개냐 고양이냐, 숫자 3이냐 아니냐.
- 주로 합성곱 신경망(CNN, Convolutional Neural Network) 사용.

6.1.1 LeNet-5



개요

- 1995년 Yann LeCun이 개발한 초기 CNN 구조.
- CNN의 시초 모델로, 오늘날 대부분의 이미지 분류 모델의 기반이 됨.
- 구조는 합성곱층(Convolution) + 하위 샘플링층(Sub-sampling / Pooling) + 완전연결층(Fully Connected Layer) 로 구성됨.

LeNet-5의 구조

단계	층 종류	출력 크기	설명
입력	32x32 이미지	-	예: 흑백 손글씨
C1	합성곱(5x5)	28x28x6	6개의 특성 맵(feature map) 생성
S2	하위 샘플링	14x14x6	크기를 절반으로 줄임
C3	합성곱(5x5)	10x10x16	16개의 특성 맵 생성
S4	하위 샘플링	5x5x16	다시 절반 축소

단계	층 종류	출력 크기	설명
C5	합성곱(5×5)	1×1×120	120개 특성 맵 생성
F6	완전연결층	84	특징 요약 및 분류 준비
출력	완전연결층	10	10개 클래스 (예: 0~9 숫자 등)

CNN이 진행될수록 공간 크기는 작아지고, 특성 맵 수는 증가한다.

현대적인 PyTorch 구현 버전

- 합성곱층(Conv) → Max Pooling → Conv → Max Pooling → Fully Connected Layer 2개 → Softmax
- 활성화 함수: ReLU 사용
- 출력층: Softmax (확률 형태로 클래스 예측)

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화
이미지	1	32×32	-	-	-
합성곱층	6	28×28	5×5	1	ReLU
최대 풀링	6	14×14	2×2	2	-
합성곱층	16	10×10	5×5	1	ReLU
최대 풀링	16	5×5	2×2	2	-
완전연결층	-	120	-	-	ReLU
완전연결층	-	84	-	-	ReLU
완전연결층	-	10	-	-	Softmax

tqdm 라이브러리

- 학습 시 진행 상태(progress bar) 를 시각적으로 보여주는 도구.
- 설치 방법:

```
pip install --user tqdm
```

또는

```
conda install -c conda-forge tqdm
```

필수 라이브러리

```
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms
from torch.autograd import Variable
from torch import optim, nn
```

```
import torch.nn.functional as F
import cv2
from PIL import Image
from tqdm import tqdm_notebook as tqdm
import random
from matplotlib import pyplot as plt
```

역할 요약

- torch / torchvision: 신경망, 데이터셋, 이미지 변환
- optim / nn: 학습용 옵티마이저, 신경망 레이어
- cv2 / PIL: 이미지 입출력
- tqdm: 학습 진행 표시
- matplotlib: 시각화

이미지 전처리 (ImageTransform 클래스)

이미지를 텐서로 변환하고, 학습/검증용으로 다르게 처리함.

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale=(0.5,1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }

    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

torchvision.transforms 개념 정리

함수	설명
transforms.Compose	여러 변환(transform)들을 순서대로 묶는 함수

함수	설명
transforms.RandomResizedCrop	입력 이미지를 랜덤한 비율과 위치로 잘라서 지정한 크기로 리사이즈 (scale=(0.5,1.0)은 50~100% 비율로 무작위 자름)
transforms.RandomHorizontalFlip	50% 확률로 이미지를 좌우 반전시킴 (데이터 증강 효과)
transforms.ToTensor	이미지를 Tensor로 변환 (픽셀값 0~255 → 0.0~1.0으로 정규화, 차원 순서도 (H×W×C → C×H×W))
transforms.Normalize(mean, std)	채널별로 평균(mean)과 표준편차(std)로 정규화 일반적으로 ImageNet의 (mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]) 사용

OpenCV는 BGR 순서로 이미지를 읽기 때문에 RGB 순서로 변환 필요.

파이썬의 `__call__` 메서드

- 클래스 내부에 `__call__` 메서드를 정의하면, 클래스 인스턴스를 함수처럼 호출할 수 있음.
- `__init__`: 인스턴스 생성 시 실행
- `__call__`: 인스턴스가 호출될 때 실행

예시:

```
transform = ImageTransform(224, mean, std)
img = transform(image, 'train')
```

`transform.__call__(image, 'train')` 이 자동으로 실행됨.

핵심 요약

구분	핵심 내용
모델 이름	LeNet-5
구조	합성곱 → 풀링 → 합성곱 → 풀링 → 완전연결층
입력 크기	32×32
특징	초기 CNN 구조, ReLU + Softmax 사용
데이터 전처리	Random Crop, Flip, ToTensor, Normalize
Normalize 값	mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225]
tqdm	학습 진행 시각화 도구
call	클래스를 함수처럼 호출 가능하게 하는 메서드

목적

고양이와 개 이미지를 불러와서

→ 훈련용 / 검증용 / 테스트용 데이터셋으로 나누는 과정을 수행하는 코드입니다.

데이터 경로 설정

```
cat_directory = r'../chap06/data/dogs-vs-cats/Cat/'
```

```
dog_directory = r'../chap06/data/dogs-vs-cats/Dog/'
```

- `r'...'`: raw string으로, 문자열 안의 `\` (역슬래시)를 이스케이프 문자로 인식하지 않도록 함.
- 각각 고양이 / 개 이미지가 저장된 폴더 경로를 나타냅니다.

이미지 파일 경로 리스트 생성

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in os.listdir(dog_directory)])
```

각 줄은 해당 폴더 내 모든 이미지 파일의 전체 경로 리스트를 만듭니다.

구성 요소별 의미

함수	설명
<code>os.listdir(path)</code>	해당 폴더 내의 모든 파일 이름을 리스트로 반환
<code>os.path.join(path, file)</code>	디렉터리 경로와 파일 이름을 결합하여 하나의 전체 경로로 만듦
<code>sorted()</code>	파일 이름 순서대로 정렬된 리스트를 반환 (순서 고정용)

예시

```
os.listdir('../data/Cat')
# ['cat.0.jpg', 'cat.1.jpg', 'cat.2.jpg']

os.path.join('../data/Cat', 'cat.0.jpg')
# '../data/Cat/cat.0.jpg'
```

고양이와 개 이미지 합치기

```
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
```

- 두 리스트(`cat_images_filepaths`, `dog_images_filepaths`)를 병합.
- *는 리스트 풀기(unpacking) 연산자.

이미지 유효성 검사

```
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]
```

- 목적: 손상된 이미지 파일이 있으면 제외하기.
- `cv2.imread(i)` → OpenCV로 이미지 파일 읽기.
- `None` 이면 읽을 수 없는 파일이므로 제외.

리스트 내포(list comprehension)

```
[i for i in range(5)]  
# 결과: [0, 1, 2, 3, 4]
```

→ 이 코드의 구조를 이미지 파일 경로에 적용한 것임.

난수 고정 (재현성 확보)

```
random.seed(42)
```

- 랜덤 동작(셔플, 난수 발생 등)의 결과를 항상 동일하게 만들기 위한 설정.
- 같은 시드(seed)를 주면 항상 동일한 순서로 섞임.

예시 (numpy)

```
import numpy as np  
  
np.random.seed(101)  
np.random.randint(1, 10, size=10)  
# array([2, 7, 8, 9, 5, 9, 6, 1, 6, 9])
```

시드를 바꾸면 다른 결과가 나옴.

다시 같은 시드를 주면 결과가 동일하게 나옴.

이미지 순서 무작위 섞기

```
random.shuffle(correct_images_filepaths)
```

- 데이터를 무작위로 섞어줌 (훈련 시 편향 방지).
- `random.seed(42)` 덕분에 항상 같은 순서로 섞임.

훈련 / 검증 / 테스트 데이터 분리

```
train_images_filepaths = correct_images_filepaths[:400]  
val_images_filepaths = correct_images_filepaths[400:-10]  
test_images_filepaths = correct_images_filepaths[-10:]
```

데이터셋	범위	개수	설명
훈련(train)	처음 400개	400개	모델 학습용
검증(validation)	400번째 ~ 끝에서 10번째 전까지	92개	하이퍼파라미터 튜닝용
테스트(test)	마지막 10개	10개	최종 성능 평가용

데이터 개수 확인

```
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

출력 결과:

```
400 92 10
```

핵심 개념 요약

구분	개념	설명
<code>os.listdir()</code>	폴더 내 파일 목록 불러오기	파일 이름 리스트 반환
<code>os.path.join()</code>	경로와 파일 이름 결합	"폴더/파일명" 형태의 전체 경로 생성
<code>sorted()</code>	리스트 정렬	파일 이름 순서 유지
리스트 내포 <code>[i for i in ...]</code>	간결한 반복문 표현	조건문과 반복문을 한 줄에 작성
<code>cv2.imread()</code>	이미지 읽기 함수	<code>None</code> 이면 파일 손상
<code>random.seed()</code>	난수 시드 고정	결과 재현성 확보
<code>random.shuffle()</code>	리스트 무작위 섞기	순서 랜덤화
데이터 분리	train / val / test	400 / 92 / 10 개로 분리

테스트 데이터셋 이미지 확인 함수

목적

테스트용 데이터셋에 있는 이미지들을 불러와서

- 이미지가 잘 불러와지는지,
 - 라벨이 올바르게 표시되는지,
 - 예측 결과가 맞는지(색상으로 표시)
- 를 한눈에 확인하기 위한 함수입니다.

함수 구조

```
def display_image_grid(images_filepaths, predicted_labels=[], cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(rows=rows, ncols=cols, figsize=(12, 6))

    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
        predicted_label = predicted_labels[i] if predicted_labels else true_label
        color = "green" if true_label == predicted_label else "red"
```

```
ax.ravel()[i].imshow(image)
ax.ravel()[i].set_title(predicted_label, color=color)
ax.ravel()[i].set_axis_off()

plt.tight_layout()
plt.show()
```

코드 단계별 개념

① `cv2.cvtColor()`

- 이미지의 색상 공간을 변환하는 함수.
- OpenCV는 이미지를 BGR(파랑-초록-빨강) 순서로 읽습니다.
- 하지만 matplotlib는 RGB(빨강-초록-파랑) 순서를 사용하기 때문에
→ `cv2.COLOR_BGR2RGB` 로 변환해야 색이 정상적으로 보입니다.

📌 예시:

```
cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

파라미터	설명
<code>image</code>	입력 이미지
<code>cv2.COLOR_BGR2RGB</code>	색상 변환 방식 (BGR → RGB)

`os.path.normpath()` + `split(os.sep)[-2]`

이미지 경로에서 폴더 이름(=정답 라벨) 을 추출하는 부분입니다.

```
true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
```

함수	설명
<code>os.path.normpath(path)</code>	경로를 정규화 (예: <code>A/B</code> 와 <code>A//B</code> , <code>A./B</code> 모두 동일하게 처리)
<code>split(os.sep)</code>	디렉터리 구분자(<code>/</code> 또는 <code>\</code>) 기준으로 경로를 분리
<code>[-2]</code>	경로의 끝에서 두 번째 요소, 즉 상위 폴더명 을 가져옴 (예: "Cat" 또는 "Dog")

예시

경로: `../chap06/data/dogs-vs-cats/Cat/cat.1.jpg`

→ `split(os.sep)` 결과: `['..', 'chap06', 'data', 'dogs-vs-cats', 'Cat', 'cat.1.jpg']`

→ `[-2]` ⇒ `'Cat'` (정답 라벨)

`predicted_label` 설정


```
predicted_label = predicted_labels[i] if predicted_labels else true_label
```

- 만약 모델의 예측 결과(`predicted_labels`)가 있다면 그 값을 사용.
- 예측 결과가 아직 없다면 → 진짜 라벨(`true_label`) 을 대신 표시.

즉,

“예측값이 있으면 사용하고, 없으면 정답 라벨을 출력한다”
는 의미입니다.

색상 구분

```
color = "green" if true_label == predicted_label else "red"
```

- 예측이 맞으면 제목 색을 초록색(`green`) 으로,
- 틀리면 빨간색(`red`) 으로 표시합니다.

이렇게 하면 이미지가 맞게 분류되었는지 시각적으로 한눈에 확인할 수 있습니다.

이미지 출력 관련 코드

코드	설명
<code>ax.ravel()</code>	<code>subplot</code> (서브그래프) 배열을 1차원으로 평탄화하여 순서대로 접근
<code>imshow(image)</code>	이미지를 표시
<code>set_title(predicted_label, color=color)</code>	예측 라벨을 색상과 함께 제목으로 표시
<code>set_axis_off()</code>	눈금선 및 축 제거 (이미지 깔끔하게 표시)
<code>plt.tight_layout()</code>	그래프 간 여백 자동 조정
<code>plt.show()</code>	그래프 출력

결과 예시

```
display_image_grid(test_images_filepaths)
```

출력 결과:

테스트 데이터셋의 이미지 10장이 표시되며,

각 이미지 위에 라벨(`Cat` , `Dog`)이 뜹니다.

- 초록색: 올바른 예측
- 빨간색: 잘못된 예측

전체 동작 흐름

- 테스트 이미지 파일 경로 목록을 받음
- 한 장씩 읽어서 RGB로 변환

- 파일 경로로부터 실제 라벨(Cat / Dog) 추출
- 예측 라벨이 있다면 비교해서 색상 결정
- 이미지 + 라벨을 subplot에 시각화

Dataset과 DataLoader의 동작 원리

이제 모델 학습을 위한 데이터셋 준비가 끝났으므로,
이제부터는 PyTorch의 Dataset 과 DataLoader 가 어떻게 데이터를 불러오는지 설명합니다.

Dataset 클래스의 기본 개념

- PyTorch에서 `torch.utils.data.Dataset` 을 상속받아 사용자 정의 데이터셋 클래스를 만듭니다.
- 이 클래스는 학습용 데이터를 메모리에 한 번에 다 올리지 않고, 필요할 때마다 이미지를 불러오는 구조입니다.

주요 메서드 2가지

메서드	설명
<code>__init__</code>	이미지 경로와 라벨, 변환(transform) 등을 초기화만 함. 실제로 데이터를 불러오지는 않음.
<code>__getitem__</code>	지정된 인덱스의 이미지를 불러오는 함수. 실제 데이터 로딩은 이때 수행됨.

DataLoader의 역할

- `Dataset` 객체에서 데이터를 한 번에 전부 불러오는 것이 아니라, 배치(batch) 단위로 나누어 불러옵니다.
- 이렇게 하면 메모리 효율을 높이고, 대규모 데이터도 처리할 수 있습니다.

DogsVsCatsDataset 클래스 정의

```
class DogsVsCatsDataset(Dataset):
    def __init__(self, file_list, transform=None, phase='train'):
        self.file_list = file_list
        self.transform = transform
        self.phase = phase

    def __len__(self):
        return len(self.file_list)

    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path)
        img_transformed = self.transform(img, self.phase)
        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
```

```
elif label == 'cat':
    label = 0
return img_transformed, label
```

핵심 개념

메서드	역할
<code>__init__</code>	파일 리스트, 변환(transform), 학습 단계(phase)를 초기화
<code>__len__</code>	전체 데이터의 개수를 반환
<code>__getitem__</code>	인덱스로 이미지를 불러오고 변환 후 (이미지, 레이블)을 반환

- `Image.open(img_path)` : 이미지를 불러옴
- `self.transform(img, self.phase)` : 전처리(리사이즈, 정규화 등) 적용
- `img_path.split('/')[-1].split('.')[0]` : 파일 이름에서 `dog` 또는 `cat` 추출
- 레이블: `dog → 1`, `cat → 0`

하이퍼파라미터 설정

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

파라미터	설명
<code>size</code>	이미지를 224×224 크기로 조정
<code>mean</code> , <code>std</code>	ImageNet 데이터의 평균과 표준편차 값 (정규화에 사용)
<code>batch_size</code>	한 번에 불러올 이미지 개수 (메모리 절약용)

학습/검증용 데이터셋 정의

```
train_dataset = DogsVsCatsDataset(
    train_images_filepaths, transform=ImageTransform(size, mean, std), phase='train'
)

val_dataset = DogsVsCatsDataset(
    val_images_filepaths, transform=ImageTransform(size, mean, std), phase='val'
)
```

- `train_dataset` : 학습용 이미지와 라벨을 담고 있음
- `val_dataset` : 검증용 이미지와 라벨을 담고 있음
- `ImageTransform` : phase에 따라 서로 다른 전처리를 적용 (`train`, `val`)

```

index = 0
print(train_dataset.__getitem__(index)[0].size()) # torch.Size([3, 224, 224])
print(train_dataset.__getitem__(index)[1])      # 0 or 1

```

→ 이미지 크기와 라벨(고양이=0, 개=1)이 출력됨.

DataLoader 정의

```

train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

```

파라미터	의미
<code>dataset</code>	볼러올 데이터셋 객체
<code>batch_size</code>	한 번에 볼러올 이미지 개수
<code>shuffle</code>	데이터를 무작위로 섞을지 여부

역할

- `DataLoader` 는 데이터를 배치 단위로 관리함.
(전체 데이터를 한 번에 메모리에 올리지 않고, 나눠서 효율적으로 처리)

배치 데이터 확인

```

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size()) # torch.Size([32, 3, 224, 224])
print(label)

```

결과 예시:

```

torch.Size([32, 3, 224, 224])
tensor([1, 1, 0, 0, 1, 1, 0, ...])

```

→ 한 번에 32장의 컬러 이미지를 불러오며,
각 이미지에 대해 개(1) 또는 고양이(0) 라벨이 지정됨.

전체 개념 흐름 요약

단계	구성요소	설명
데이터 정의	<code>DogsVsCatsDataset</code>	이미지 로딩 + 레이블 지정
전처리 설정	<code>ImageTransform</code>	resize, normalize 등
하이퍼파라미터	size, mean, std, batch_size	데이터 전처리 기준
DataLoader	<code>train_dataloader</code> , <code>val_dataloader</code>	배치 단위 데이터 로딩

단계	구성요소	설명
출력 확인	<code>inputs.size()</code> , <code>labels</code>	데이터가 올바르게 불러와졌는지 확인

LeNet 신경망 모델 개념 정리

LeNet 클래스 정의

```
class LeNet(nn.Module):
    def __init__(self):
        super(LeNet, self).__init__()
```

- `nn.Module` : PyTorch의 모든 신경망 모델이 상속해야 하는 기본 클래스
- `super()` : 부모 클래스(`nn.Module`)의 초기화 메서드를 호출하여 기본 기능을 상속받음

LeNet의 구성 계층

```
self.cnn1 = nn.Conv2d(3, 16, kernel_size=5, stride=1, padding=0)
self.relu1 = nn.ReLU()
self.maxpool1 = nn.MaxPool2d(kernel_size=2)

self.cnn2 = nn.Conv2d(16, 32, kernel_size=5, stride=1, padding=0)
self.relu2 = nn.ReLU()
self.maxpool2 = nn.MaxPool2d(kernel_size=2)

self.fc1 = nn.Linear(32*53*53, 512)
self.relu5 = nn.ReLU()
self.fc2 = nn.Linear(512, 2)
self.output = nn.Softmax(dim=1)
```

각 층의 역할

층	이름	역할
Conv2d	합성곱층	이미지 특징 추출 (필터 적용)
ReLU	활성화 함수	비선형성 부여 (음수값 제거)
MaxPool2d	풀링층	특징 맵의 크기 축소
Linear	완전 연결층	최종 분류 (고양이/개)
Softmax	출력층	클래스 확률로 변환

Forward 함수 (순전파)

```
def forward(self, x):
    out = self.cnn1(x)
    out = self.relu1(out)
```

```

out = self.maxpool1(out)
out = self.cnn2(out)
out = self.relu2(out)
out = self.maxpool2(out)
out = out.view(out.size(0), -1)
out = self.fc1(out)
out = self.fc2(out)
out = self.output(out)
return out

```

- 순전파(Forward Propagation): 입력 데이터를 순서대로 각 층에 통과시키는 과정
- `view(out.size(0), -1)` : CNN 출력(다차원)을 1차원 벡터로 변환하여 완전연결층에 전달

합성곱(Conv2d) 출력 크기 공식

기호	의미
W	입력 크기 (Input size)
F	커널 크기 (Filter size)
P	패딩 (Padding size)
S	스트라이드 (Stride)

MaxPooling 출력 크기 공식

- 예: 입력 크기 24, 커널 크기 2 → 출력 크기 12

torchsummary 사용

```
pip install torchsummary
```

```

from torchsummary import summary
summary(model, input_size=(3, 224, 224))

```

summary() 함수

- 모델의 구조, 각 층의 출력 크기, 파라미터 수를 한눈에 확인 가능
- `(3, 224, 224)` 는 입력 이미지의 채널 수, 높이, 너비

torchsummary 출력 예시

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216

- Output Shape: 출력 텐서의 크기 (배치, 채널, 높이, 너비)
- Param #: 해당 계층에서 학습해야 할 파라미터 개수

학습 가능한 파라미터 수 확인

```
def count_parameters(model):  
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

- `model.parameters()` : 모델 내의 모든 가중치(weight)와 편향(bias)을 반환.
- `requires_grad=True` : 학습(gradient 계산 및 업데이트)에 참여하는 파라미터만 포함.
- 결과 예시:

```
The model has 46,038,242 trainable parameters
```

→ 모델이 약 4천6백만 개의 파라미터를 학습해야 한다는 뜻.

개념 요약

- CNN 모델의 복잡도를 나타내는 지표.
- 파라미터 수가 많을수록 연산량이 커지고 학습 시간이 길어진다.
- 따라서 예제에서는 전체 이미지가 아닌 일부만 사용해 학습한다.

옵티마이저와 손실 함수 정의

```
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)  
criterion = nn.CrossEntropyLoss()
```

SGD (Stochastic Gradient Descent)

- `model.parameters()` : 업데이트할 파라미터(가중치, 편향)
- `lr` (learning rate) : 한 번에 얼마나 크게 값이 조정될지 결정 (학습 속도)
- `momentum` : 이전 변화 방향을 참고하여 진동(oscillation)을 줄이고 더 빠른 수렴을 유도

손실 함수 (Loss Function)

- `nn.CrossEntropyLoss()`
 - 분류 문제에서 자주 사용.
 - 모델 출력(예측 확률)과 실제 정답(label)의 차이를 계산.

모델과 손실 함수를 CPU/GPU로 이동

```
model = model.to(device)  
criterion = criterion.to(device)
```

- PyTorch는 GPU 연산을 명시적으로 지정해야 함.
- 예제에서는 CPU를 사용하지만, GPU가 있다면 `'cuda'` 로 설정 가능.

모델 학습 함수 정의

핵심 구조

```
for epoch in range(num_epoch):
    for phase in ['train', 'val']:
        if phase == 'train':
            model.train()  # 학습 모드
        else:
            model.eval()   # 평가 모드
```

- **train**: 가중치를 업데이트 (gradient 계산, optimizer step)
- **val**: 가중치 고정, 모델의 성능 평가만 수행

학습 단계 (Forward → Backward)

1. 입력 & 정답 이동

```
inputs = inputs.to(device)
labels = labels.to(device)
```

2. 기울기 초기화

```
optimizer.zero_grad()
```

3. 순전파 (Forward)

```
outputs = model(inputs)
loss = criterion(outputs, labels)
```

4. 역전파 (Backward)

```
loss.backward()
optimizer.step()
```

→ 손실을 기준으로 파라미터를 업데이트

5. 정확도 계산

```
_, preds = torch.max(outputs, 1)
epoch_corrects += torch.sum(preds == labels.data)
```

출력 (각 epoch 마다)

```
Epoch 1/10
train Loss: 0.6971 Acc: 0.4875
```



```
val Loss: 0.7125 Acc: 0.4457
```

손실 함수의 **reduction** 개념

```
criterion = nn.CrossEntropyLoss(reduction='mean')
```

- **reduction='mean'** → 배치 단위로 나온 손실들의 평균을 계산.
- **sum** 으로 바꾸면 모든 손실을 더함.
- **none** 이면 각 데이터의 손실을 개별적으로 반환.

💡 즉,

`loss.item() * inputs.size(0)` 은 평균 손실을 다시 전체 배치 크기로 환산하기 위해 사용됨.

모델 학습 실행

```
num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

- 10 epoch 동안 모델을 학습.
- **train_model()** 함수 내부에서:
 - 각 epoch 마다 **train** → **val** 수행.
 - 최고 정확도(**best_acc**) 모델 가중치를 저장.

테스트 단계 개요

- 학습이 끝난 모델(LeNet)을 테스트 데이터셋에 적용하여 예측 정확도와 결과를 확인하는 과정입니다.
- 학습 시 사용되지 않은 새로운 데이터로 모델의 성능을 검증합니다.
- 테스트 시에는 학습처럼 gradient 계산이 필요 없으므로 **torch.no_grad()** 와 **model.eval()** 을 사용합니다.

테스트 함수 구조

```
with torch.no_grad():
    for test_path in tqdm(test_images_filepaths):
        img = Image.open(test_path)
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
```

```
outputs = model(img)
preds = F.softmax(outputs, dim=1)[: , 1].tolist()
```

주요 개념

코드	의미
<code>torch.no_grad()</code>	학습 시 필요한 gradient 계산을 생략 (메모리 절약, 속도 향상)
<code>model.eval()</code>	평가 모드로 변경 (Dropout, BatchNorm 등 비활성화)
<code>unsqueeze(0)</code>	이미지에 배치 차원 추가 (1장이라도 [1, C, H, W] 형태로)
<code>softmax(outputs, dim=1)</code>	각 클래스에 대한 확률 계산 (dim=1은 class dimension)
<code>[:, 1]</code>	클래스 1 (dog)의 확률만 선택
<code>.tolist()</code>	텐서를 Python 리스트로 변환

예측 결과 저장

```
res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})
res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)
res.to_csv('./chap06/data/LeNet', index=False)
```

요약:

- 예측 결과(`id` , `label`)를 데이터프레임으로 저장.
- `label` = "dog 확률값".
- 이후 CSV 파일로 출력 → 예측 결과를 다른 곳에서도 활용 가능.

torch.unsqueeze() 개념

`unsqueeze(dim)` 은 텐서에 새 차원을 추가하는 함수입니다.

예시:

- 원래: `(3,)` → `unsqueeze(0)` → `(1,3)`
- 2D → `(2,2)` 일 때 `unsqueeze(0)` → `(1,2,2)`
즉, "배치 차원 추가"의 의미입니다.

softmax와 인덱스 선택

```
F.softmax(outputs, dim=1)[: , 1].tolist()
```

- softmax를 적용해 `[고양이 확률, 개 확률]` 로 만든 뒤,

두 번째 값([, 1])인 **개(dog)**의 확률만 선택.

예측 결과 예시

```
res.head(10)
```

id	label
115	0.412220
116	0.422235
194	0.545277
99	0.590979

✓ label > 0.5 → dog

✓ label < 0.5 → cat

시각화

```
def display_image_grid(images_filepaths, predicted_labels, cols=5):
    class_ = {0:'cat', 1:'dog'}
    ...
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        ...
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
```

개념:

- `cv2.imread()` 로 이미지를 읽고,
- `cv2.cvtColor()` 로 색상 보정,
- `imshow()` 로 표시하며,
- 예측 결과('cat', 'dog')를 제목으로 출력.

결과적으로 테스트 이미지와 예측 라벨을 한눈에 확인할 수 있습니다.

결과 해석

- 예측 정확도는 약 64% 정도로 높지 않음.
- 이유:
 - 전체 데이터 중 일부만 사용했기 때문.
 - CPU 환경에서 훈련 → 속도와 성능 제한.

- 전체 데이터로, GPU 환경에서 학습하면 더 나은 정확도 기대 가능.

다음 단계

- 여기까지는 LeNet 구조를 학습.
- 이후 장에서는 AlexNet 등 더 깊은 CNN 모델을 학습함.
- AlexNet은 LeNet과 유사한 구조이지만 더 복잡하고 강력한 모델.

6.1.2 AlexNet 개념 정리

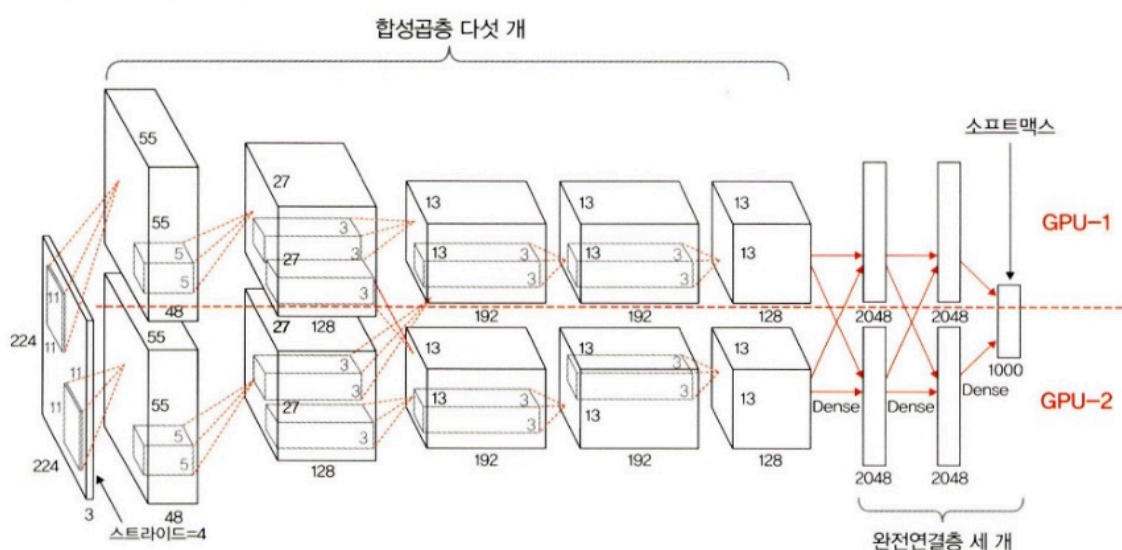
개요

- AlexNet은 ImageNet 영상 데이터베이스를 기반으로 한 **이미지 인식 대회(ILSVRC 2012)**에서 우승한 CNN(Convolutional Neural Network) 구조입니다.
- 이 모델은 딥러닝 시대를 연 대표적인 신경망으로, 이후 등장한 많은 모델(AlexNet, VGG, ResNet 등)의 기반이 되었습니다.

CNN 기본 구조 복습

- CNN은 이미지 데이터를 다루기 때문에 **3차원 구조(Width × Height × Depth)**를 가집니다.
 - **Width** : 이미지의 너비
 - **Height** : 이미지의 높이
 - **Depth** : 채널 수 (보통 RGB → 3)
- 합성곱(Convolution)을 거치면 **특징 맵(Feature Map)**이 만들어지고, 이 맵의 깊이는 점점 변합니다.

AlexNet의 구성



- AlexNet은 **총 8개의 층(layer)**으로 구성됨:

- 5개의 합성곱 층(Convolutional Layers)
- 3개의 완전 연결층(Fully Connected Layers)
- 마지막 출력층은 Softmax를 사용하여 1000개 카테고리를 분류합니다.
- *활성화 함수(Activation Function)**로 **ReLU(Rectified Linear Unit)**를 사용하여 학습 속도를 크게 향상시켰습니다.

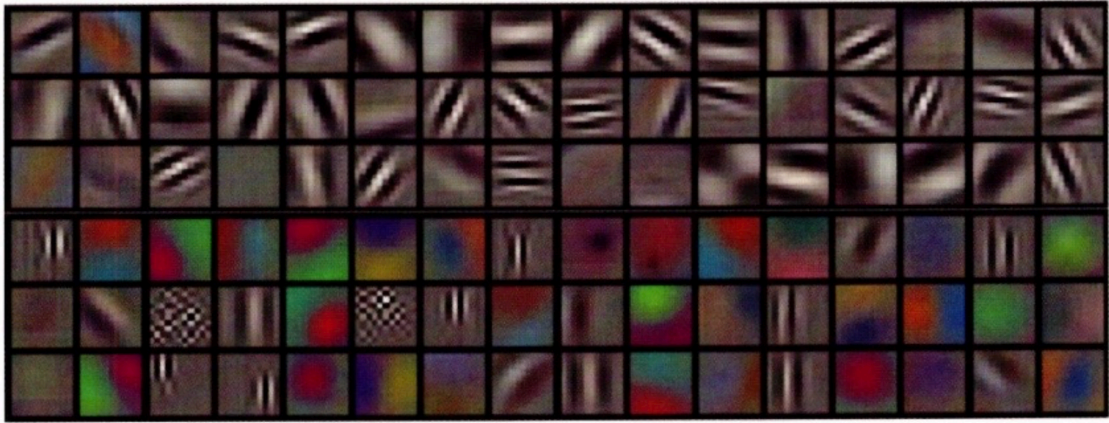
AlexNet 구조 세부 내용

계층 유형	출력 특성맵	커널 크기	스트라이드	활성화 함수
입력	227×227×3	-	-	-
Conv1	55×55×96	11×11	4	ReLU
Max Pool1	27×27×96	3×3	2	-
Conv2	27×27×256	5×5	1	ReLU
Max Pool2	13×13×256	3×3	2	-
Conv3	13×13×384	3×3	1	ReLU
Conv4	13×13×384	3×3	1	ReLU
Conv5	13×13×256	3×3	1	ReLU
Max Pool3	6×6×256	3×3	2	-
FC1	4096	-	-	ReLU
FC2	4096	-	-	ReLU
Output	1000	-	-	Softmax

GPU 병렬 처리

- AlexNet은 당시 컴퓨팅 한계를 극복하기 위해 두 개의 GPU로 병렬 학습을 수행했습니다.
 - GPU-1: 주로 형태(Shape), 패턴 등 색상과 무관한 특징을 학습
 - GPU-2: 주로 색상(Color) 관련 특징을 학습
- 아래 그림처럼 서로 다른 특징을 학습합니다.
 - GPU-1은 흑백 패턴, GPU-2는 색상 패턴을 주로 학습

▼ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



첫 번째 합성곱층의 특징

- 커널 크기: $11 \times 11 \times 3$
- 스트라이드: 4
- 출력 맵 수: 96개 ($55 \times 55 \times 96$)
- 초반에는 단순한 경계선(edge), 색상 대비, 패턴 등을 학습합니다.
- 이후 층으로 갈수록 더 복잡한 형태(예: 얼굴, 물체 형태 등)를 학습합니다.

특징 요약

- ReLU 사용 → 빠른 학습 가능
- Dropout 적용 → 과적합 방지
- GPU 병렬 처리 → 대용량 이미지 학습 가능
- ImageNet에서 큰 성능 향상 달성 (Top-5 error 약 15.3%)

라이브러리 호출

AlexNet을 구현하기 위해 필요한 기본 PyTorch 및 이미지 처리 라이브러리를 불러옵니다.

주요 라이브러리

라이브러리	용도
<code>torch</code> , <code>torchvision</code>	딥러닝 기본 기능과 모델 구조 구성
<code>torch.utils.data</code>	<code>DataLoader</code> , <code>Dataset</code> 을 이용한 데이터 관리
<code>torchvision.transforms</code>	이미지 전처리(크기 조정, 정규화 등)
<code>torch.nn</code> , <code>torch.nn.functional</code>	신경망 계층, 활성화 함수, 손실 함수
<code>cv2</code> , <code>PIL.Image</code>	이미지 파일 읽기 및 변환
<code>tqdm</code>	학습 진행률 표시
<code>device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")</code>	GPU 사용 설정

AlexNet 네트워크 구조 개요

- 입력 이미지 크기: `227×227×3`
- 합성곱층(Convolution): 5개
- 완전연결층(Fully Connected): 3개
- 마지막 출력층: Softmax (예제에서는 2개 클래스: Cat/Dog)
- 각 합성곱층 뒤에는 `ReLU` 활성화 함수 사용
- 일부 층에는 MaxPooling이 적용되어 크기를 점점 줄임

데이터 전처리

데이터를 모델에 맞게 변환하는 ImageTransform 클래스를 정의합니다.

구조

```
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([...]),
            'val': transforms.Compose([...])
        }
    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

주요 기능

구분	변환 내용
train	RandomResizeCrop, RandomHorizontalFlip → 데이터 다양화(augmentation)
val	Resize, CenterCrop → 평가용 표준화
공통	ToTensor(), Normalize(mean, std)

데이터 로드 및 분리

고양이/강아지 이미지를 훈련, 검증, 테스트 데이터로 나눔.

과정

1. 경로 설정:

```
cat_directory = '../chap06/data/dogs-vs-cats/Cat/'
dog_directory = '../chap06/data/dogs-vs-cats/Dog/'
```

2. 파일 경로 리스트 생성 (`os.listdir`, `cv2.imread` 로 이미지 유효성 검사)

3. 셔플 후 분리:

- train: 400장
- val: 92장
- test: 10장

💡 실제 AlexNet은 600만 개 파라미터를 사용하므로, 이렇게 적은 데이터로는 성능이 낮지만 실습 예시로 사용.

커스텀 데이터셋 정의

PyTorch의 `Dataset` 클래스를 상속받아 고양이/개 이미지와 레이블을 함께 반환하는 Dataset 생성.

주요 메서드

메서드	설명
<code>__init__</code>	파일 경로, 전처리(transform), 단계(train/val) 지정
<code>__len__</code>	전체 데이터 개수 반환
<code>__getitem__</code>	이미지와 레이블을 불러와 전처리 후 반환

```
if label == 'dog': label = 1
elif label == 'cat': label = 0
```

전처리 변수 설정

AlexNet에 맞는 이미지 크기와 정규화 파라미터를 지정합니다.

변수	의미
<code>size = 256</code>	입력 이미지 크기
<code>mean = (0.485, 0.456, 0.406)</code>	RGB 평균값
<code>std = (0.229, 0.224, 0.225)</code>	RGB 표준편차
<code>batch_size = 32</code>	한 번에 처리할 이미지 수

훈련/검증/테스트 데이터셋 생성

앞서 만든 클래스와 변환기를 이용해 Dataset 객체 생성:

```
train_dataset = DogvsCatDataset(train_images_filepaths, transform=ImageTransform(size,
mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths, transform=ImageTransform(size, m
ean, std), phase='val')
test_dataset = DogvsCatDataset(test_images_filepaths, transform=ImageTransform(size,
mean, std), phase='val')
```

데이터 크기 확인

출력 예시:


```
torch.Size([3, 256, 256]) # (채널, 너비, 높이)
1                          # 레이블 (1=dog)
```

DataLoader로 메모리에 불러오기

데이터를 배치 단위로 로드하고, 학습 시 자동으로 셔플되게 설정.

```
train_dataloader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_dataloader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_dataloader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```

데이터 확인

```
inputs, label = next(iter(train_dataloader))
print(inputs.size()) # torch.Size([32, 3, 256, 256])
print(label)
```

라이브러리 불러오기

딥러닝 모델 구현에 필요한 라이브러리를 임포트합니다.

- `torch`, `torchvision` : PyTorch 핵심 라이브러리
- `torch.utils.data` : 데이터 로더 (Dataset, DataLoader)
- `torchvision.transforms` : 데이터 전처리(Resize, Crop, Normalize 등)
- `torch.nn`, `torch.nn.functional` : 신경망 구성 요소 (층, 활성화함수 등)
- `cv2`, `os`, `random` : 이미지 입출력 및 파일 관리
- `PIL.Image` : 이미지 처리
- `tqdm_notebook` : 학습 진행률 표시
- `device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")`
→ GPU가 있으면 CUDA 사용, 없으면 CPU 사용

데이터 전처리 정의

이미지를 모델에 입력하기 전 형태를 바꾸는 과정입니다.

```
class ImageTransform():
```

- `train` 단계:
 - RandomResizedCrop, RandomHorizontalFlip → 데이터 다양화 (augmentation)
 - ToTensor, Normalize(mean, std) → 텐서로 변환 및 정규화
- `val` 단계:

- Resize → CenterCrop → ToTensor → Normalize

데이터 경로 설정 및 분할

- Cat/Dog 이미지 경로 지정
- `os.listdir()` 로 파일 경로를 가져오고 `cv2.imread()` 로 손상된 이미지 제외
- 데이터를 train(400장) / val(92장) / test(10장) 으로 분리
→ 전체 502개 이미지 사용

커스텀 데이터셋 정의

`torch.utils.data.Dataset` 을 상속한 클래스

```
class DogsVsCatsDataset(Dataset):
```

- `__getitem__` : index를 받아 해당 이미지를 로드하고 변환(transform)을 적용
→ 라벨(label)은 파일 경로 이름('dog', 'cat')에서 추출
(`dog` = 1, `cat` = 0)
- `__len__` : 전체 이미지 수 반환

전처리용 변수 정의

```
size = 256  
mean = (0.485, 0.456, 0.406)  
std = (0.229, 0.224, 0.225)  
batch_size = 32
```

→ `ImageNet` 에서 사용된 평균과 표준편차 값을 그대로 사용

→ AlexNet 입력 크기에 맞게 256x256으로 설정

Dataset & DataLoader 생성

```
train_dataset = DogsVsCatsDataset(...)  
train_dataloader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

→ 데이터를 미니배치(batch) 단위로 나눠 GPU 메모리에 올림

→ 학습, 검증, 테스트용 각각의 DataLoader 생성

출력 확인:

`torch.Size([32, 3, 256, 256])` → (배치, 채널, 높이, 너비)

AlexNet 모델 정의

```
class AlexNet(nn.Module):
```

- 특징 추출부 (`self.features`) :
 - Conv2d + ReLU + MaxPool 조합을 5번 반복
- Adaptive Average Pooling (`self.avgpool`) :
 - 입력 크기에 맞게 평균 풀링 크기를 자동 조정
- 분류기 (`self.classifier`) :
 - 4096 → 512 → 2 (출력층, 두 클래스)
 - Dropout으로 과적합 방지
- forward() :
 - 입력 → feature 추출 → pooling → flatten → classifier → 출력

모델 객체 생성 및 손실 함수 정의

```
model = AlexNet()
model.to(device)
optimizer = optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

- `SGD` : 확률적 경사하강법
- `CrossEntropyLoss` : 분류 문제용 손실 함수

모델 구조 확인

```
from torchsummary import summary
summary(model, input_size=(3, 256, 256))
```

→ 레이어별 파라미터 수와 출력 크기를 한눈에 확인 가능

학습 함수 정의

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
```

- 훈련(`train`) / 검증(`val`) 모드 구분
- 각 epoch마다 손실(loss)과 정확도(acc) 계산
- `torch.set_grad_enabled()` 로 train일 때만 gradient 계산
- `optimizer.zero_grad()` , `loss.backward()` , `optimizer.step()` 으로 파라미터 업데이트
- 결과 출력:
 - `Loss` , `Acc` , `시간` 출력

모델 학습 실행

```
num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

출력 예시:

```
Epoch 1/10
train Loss: 0.6929 Acc: 0.4950
val Loss: 0.6926 Acc: 0.5109
```

→ 데이터 수가 적고 CPU 환경이므로 성능이 낮게 나옴.

→ 더 많은 데이터와 GPU 사용 시 성능 향상 가능.

예측 준비

- pandas 불러오기: 예측 결과를 CSV로 저장하기 위함
- id_list, pred_list 초기화: 각각 이미지 번호와 예측 결과 저장
- torch.no_grad(): 추론(inference) 시 기울기 계산 비활성화 (메모리 절약)

```
for test_path in tqdm(test_images_filepaths):
    img = Image.open(test_path)
    _id = test_path.split('/')[-1].split('.')[1] # ex) dog.113.jpg → 113
    transform = ImageTransform(size, mean, std) # 테스트 이미지 변환
```

- 이미지 전처리(transform) : 학습 시와 같은 크기 및 정규화(mean, std) 적용
- 모델 예측

```
model.eval()
outputs = model(img)
preds = F.softmax(outputs, dim=1)[0, 1].tolist()
```

→ softmax로 클래스 확률 계산 (여기선 `[0]=cat`, `[1]=dog`)

- 결과 저장

```
res = pd.DataFrame({'id': id_list, 'label': pred_list})
res.to_csv('./alexnet.csv', index=False)
```

→ 결과를 alexnet.csv로 저장

 CSV 결과 예시

id	label
207	0.493234
99	0.491417

→ label값이 0.5보다 크면 dog, 작으면 cat

시각화

- class 정의

```
class_ = {0:'cat', 1:'dog'}
```

- 랜덤 이미지 선택 + 예측 결과 표시

```
a = random.choice(res['id'].values)
label = res.loc[res['id']==a, 'label'].values[0]
if label > 0.5: label = 1
else: label = 0
```

- cv2.imshow()로 이미지 표시 + 제목에 예측 클래스(cat/dog) 표시

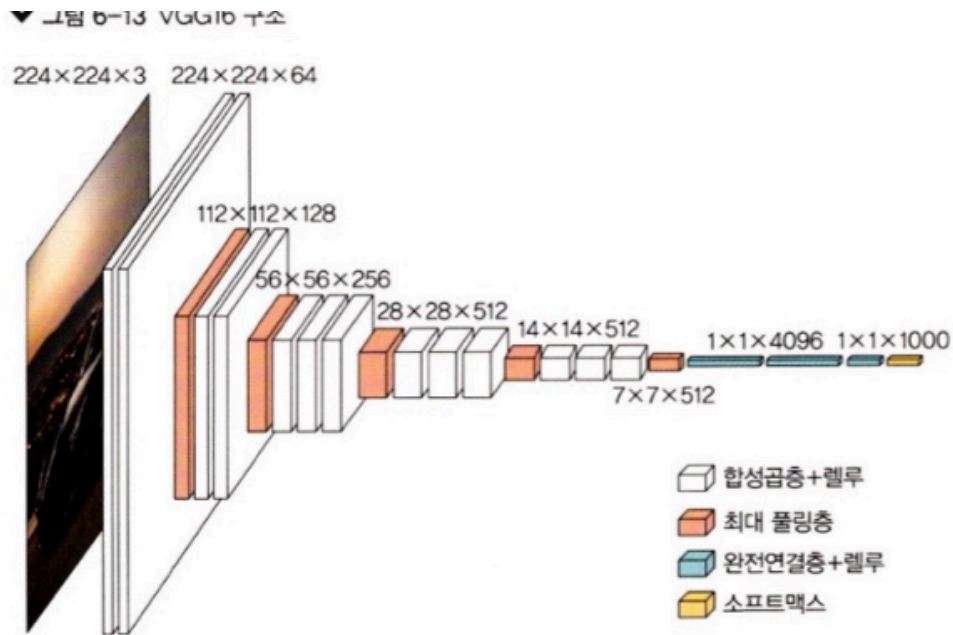
결과 분석

- 대부분 "cat"으로 분류됨 → 예측 성능 낮음
- 원인:
 - 데이터셋의 양이 적음
 - 파라미터 조정(learning rate, batch size 등) 미흡
 - Epoch 수도 적음 (10회)
- 따라서 데이터 양을 늘리거나, 하이퍼파라미터 튜닝을 통해 개선 가능

6.1.3 VGGNet 개요

- 제안자: Karen Simonyan, Andrew Zisserman (2015 ICLR 논문 발표)
- 논문명: "Very Deep Convolutional Networks for Large-Scale Image Recognition"
- 목적: 합성곱층(Convolution layer)을 깊게 쌓았을 때 성능이 어떻게 변화하는지 연구하기 위해 설계됨.
- 특징:
 - 모든 합성곱 커널 크기를 3×3으로 고정 (작은 필터 여러 개를 겹쳐 깊이를 증가).
 - Stride = 1, Padding = same (출력 크기 유지).
 - Max pooling 크기는 2×2, stride는 2로 설정.
 - 모든 합성곱층에는 ReLU 활성화 함수 사용.

VGG 구조 (특히 VGG16)



- VGG16은 이름처럼 ****16개의 층(layer)****으로 구성됨.
- 총 파라미터 수: 약 1억 3천만 개 (≈ 133 million)
- 층 구성 예시:
 - Conv(3×3) → Conv(3×3) → Pool(2×2)
 - Conv(3×3) → Conv(3×3) → Pool(2×2)
 - Conv(3×3) → Conv(3×3) → Conv(3×3) → Pool(2×2)
 - Conv(3×3) × 3 → Pool(2×2)
 - FC(4096) → FC(4096) → Softmax(1000 classes)
- 즉, 이미지 크기 224×224 → 점점 축소되어 7×7까지 줄고 → 완전연결층으로 연결됨.

합성곱(Convolution) 구조의 의미

- 3×3 필터를 여러 번 사용하면 더 큰 수용영역(receptive field) 효과를 얻을 수 있음.
예: 3×3 → 3×3 → 3×3은 실제로 7×7 효과.
- 파라미터 수는 줄이고, 비선형성(ReLU)을 더 자주 적용하여 성능 향상.

VGG의 장단점

항목	내용
장점	단순하고 규칙적인 구조로 다른 네트워크 설계의 기반이 됨 (예: ResNet, MobileNet 등)
단점	파라미터 수가 너무 많아 훈련 및 저장 비용이 큼

Shallow Copy vs Deep Copy (객체 복사 개념)

얕은 복사 (Shallow Copy)

- `copy.copy()` 또는 단순 할당(=) 사용 시.
- 원본과 복사본이 같은 메모리 참조를 공유함.
- 복사본을 수정하면 원본도 같이 바뀜.

```
import copy
original = [1, 2, 3]
copy_o = original
copy_o[2] = 10
print(original) # [1, 2, 10]
```

얕은 복사라서 original도 함께 변경됨.

깊은 복사 (Deep Copy)

- `copy.deepcopy()` 사용.
- 완전히 새로운 메모리 공간에 객체를 복사.
- 복사본을 수정해도 원본에 영향 없음.

```
import copy
original = [[1, 2], 3]
copy_o = copy.deepcopy(original)
copy_o[0][1] = 100
print(original) # [[1, 2], 3]
print(copy_o) # [[1, 100], 3]
```

깊은 복사는 서로 독립적인 메모리 공간을 사용함.

메모리 구조 비교

구분	메모리 공간 수	원본 변경 영향
얕은 복사 (shallow copy)	1개	O (공유됨)
깊은 복사 (deep copy)	2개	X (독립적)

PyTorch 실습 준비

VGG 모델 구현을 위해 필요한 라이브러리:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets
```

그리고 GPU 사용 여부 확인:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

VGG 모델 개념

- VGGNet은 매우 깊은 CNN 구조로, 작은 필터(3×3)와 반복된 계층을 사용하는 것이 특징입니다.
- 모델 이름에 따라 깊이가 달라요:
 - VGG11, VGG13, VGG16, VGG19 → 합성곱 층(Convolution layer)의 개수에 따라 구분됩니다.
- 각 층 구성은 `config` 리스트로 정의되며,
숫자 = 합성곱 채널 수, 'M' = MaxPooling 층을 의미합니다.

예:

```
vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512, 'M']
```

VGG 클래스 정의

(1) `__init__` 구성

```
class VGG(nn.Module):
    def __init__(self, self, features, output_dim):
        super().__init__()
        self.features = features
        self.avgpool = nn.AdaptiveAvgPool2d(7)
        self.classifier = nn.Sequential(
            nn.Linear(512 * 7 * 7, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(4096, output_dim)
        )
```

- `features` : 합성곱(Conv) 부분을 의미 (특징 추출)
- `classifier` : 완전 연결층 (분류 부분)
- `avgpool` : Adaptive Average Pooling으로 크기를 고정 (입력 이미지 크기 상관없이 처리 가능)

(2) `forward` 함수

```
def forward(self, x):
    x = self.features(x)
    x = self.avgpool(x)
    h = x.view(x.shape[0], -1)
```



```
x = self.classifier(h)
return x, h
```

- 합성곱 → 풀링 → 펼치기(view) → 분류 단계로 흐름이 이어집니다.

get_vgg_layers() – VGG 계층 정의 함수

```
def get_vgg_layers(config, batch_norm):
    layers = []
    in_channels = 3

    for c in config:
        assert c == 'M' or isinstance(c, int)
        if c == 'M':
            layers += [nn.MaxPool2d(kernel_size=2)]
        else:
            conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding=1)
            if batch_norm:
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)]
            else:
                layers += [conv2d, nn.ReLU(inplace=True)]
            in_channels = c
    return nn.Sequential(*layers)
```

- 'M' 이면 MaxPooling 적용
- 정수면 Conv2d → (BatchNorm) → ReLU 적용
- assert 문은 잘못된 설정을 방지하는 에러 방어 장치
- 결과적으로 Sequential 객체로 모든 계층을 묶어 반환

Batch Normalization

- 입력 데이터의 분포를 평균 0, 표준편차 1로 정규화하여 학습 안정화
- 장점:
 - 학습 속도 향상
 - 과적합 방지
 - 초기값(Weight)에 덜 민감

모델 생성

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
model = VGG(vgg11_layers, output_dim=2)
```

- batch_norm=True 로 설정 시 Batch Normalization이 포함됨

- `output_dim=2` → 예: 고양이 vs 개 (2개 클래스)

사전 훈련된 VGG 모델 사용 (Pretrained Model)

```
import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained=True)
```

- `vgg11_bn` → 배치 정규화(Batch Normalization) 버전 VGG11
- `pretrained=True` → ImageNet으로 미리 학습된 가중치 사용
(즉, 처음부터 학습하지 않아도 됨)

데이터 전처리 (Data Preprocessing)

(1) 학습용 전처리 (`train_transforms`)

```
train_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomRotation(5),
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

- 이미지 크기 조정 (256x256)
- 5도 회전, 좌우 반전 (데이터 증강)
- Tensor 변환
- 정규화 (ImageNet 평균, 표준편차 사용)

(2) 테스트용 전처리 (`test_transforms`)

```
test_transforms = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                        std=[0.229, 0.224, 0.225])
])
```

- 테스트 데이터는 랜덤 변형 없이, 크기 조정과 정규화만 수행

`ImageFolder` 로 데이터 불러오기

```
train_path = '../chap06/data/catanddog/train'
test_path = '../chap06/data/catanddog/test'
```

```
train_dataset = torchvision.datasets.ImageFolder(train_path, transform=train_transforms)
test_dataset = torchvision.datasets.ImageFolder(test_path, transform=test_transforms)
```

- `ImageFolder` 는 폴더 이름을 라벨로 자동 인식함

예:

```
train/
├── cat/
└── dog/
```

VGG 모델 학습 전체 개념 정리

데이터셋 분할 (`data.random_split`)

- 역할: 훈련용 데이터셋을 훈련(train) 과 검증(validation) 으로 나눕니다.
- 예시 코드

```
VALID_RATIO = 0.9
n_train_examples = int(len(train_dataset) * VALID_RATIO)
n_valid_examples = len(train_dataset) - n_train_examples
train_data, valid_data = data.random_split(train_dataset,
                                           [n_train_examples, n_valid_examples])
```

- 개념 요약
 - 전체 데이터의 90% → 학습용
 - 10% → 검증용
 - 무작위로 나누기 때문에 항상 동일하지 않음.

검증 데이터 전처리 (`deepcopy` & transform)

- 검증 데이터도 훈련 데이터와 구조는 같지만,
학습용 변형(예: `RandomRotation`)은 제거하고 테스트용 transform만 적용.

```
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms
```

데이터 수 확인

- 각각 데이터셋에 포함된 이미지 수를 출력:

```
print(len(train_data), len(valid_data), len(test_dataset))
```

예시 결과:

```
Number of training examples: 476
Number of validation examples: 53
Number of testing examples: 12
```

미니배치 데이터 로더 (`DataLoader`)

- 역할: 데이터를 메모리에 한 번에 모두 올리지 않고 batch 단위로 나눠서 로딩
- 코드

```
BATCH_SIZE = 128
train_iterator = data.DataLoader(train_data, shuffle=True, batch_size=BATCH_SIZE)
valid_iterator = data.DataLoader(valid_data, batch_size=BATCH_SIZE)
test_iterator = data.DataLoader(test_dataset, batch_size=BATCH_SIZE)
```

- `shuffle=True`: 매 epoch마다 데이터 순서를 섞어서 과적합 방지

옵티마이저와 손실함수 설정

- Adam Optimizer : 파라미터 자동 조정 최적화 기법
- CrossEntropyLoss : 분류(classification) 문제용 손실 함수

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
```

모델 정확도 측정 함수

```
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim=True) # 가장 높은 확률의 클래스 선택
    correct = top_pred.eq(y.view_as(top_pred)).sum() # 예측이 맞은 개수 계산
    acc = correct.float() / y.shape[0] # 전체 중 맞은 비율
    return acc
```

- `argmax(1)` → 각 이미지별로 가장 높은 클래스 선택
- `eq()` → 예측값과 실제값 비교
- `view_as()` → 텐서 크기 맞추기

모델 학습 함수 (`train`)

```
def train(model, iterator, optimizer, criterion, device):
    model.train() # 학습 모드
```

```

epoch_loss = 0
epoch_acc = 0
for (x, y) in iterator:
    x, y = x.to(device), y.to(device)
    optimizer.zero_grad()
    y_pred = model(x)
    loss = criterion(y_pred, y)
    acc = calculate_accuracy(y_pred, y)
    loss.backward()
    optimizer.step()
    epoch_loss += loss.item()
    epoch_acc += acc.item()
return epoch_loss / len(iterator), epoch_acc / len(iterator)

```

- forward → loss 계산 → backward → optimizer 업데이트
- 평균 손실과 정확도 반환

모델 검증 함수 (**evaluate**)

- 학습과 거의 동일하지만 `model.eval()` + `torch.no_grad()` 사용
- 학습 중이 아닌 평가 단계에서는 gradient 계산 비활성화 → 속도 향상

학습 시간 측정 (**epoch_time**)

```

def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
    elapsed_secs = int(elapsed_time - elapsed_mins * 60)
    return elapsed_mins, elapsed_secs

```

전체 학습 루프

```

EPOCHS = 5
best_valid_loss = float('inf')

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), '../data/VGG-model.pt')

```

```

end_time = time.monotonic()
epoch_mins, epoch_secs = epoch_time(start_time, end_time)

print(f'Epoch: {epoch+1:02} | Time: {epoch_mins}m {epoch_secs}s')
print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc*100:.2f}%')
print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc: {valid_acc*100:.2f}%')

```

- 매 epoch마다:
 - 학습/검증 손실, 정확도 계산
 - 검증 손실이 최소일 때 모델 저장
 - 시간 및 결과 출력

최종 결과 예시

```

Epoch: 01 | Epoch Time: 8m 27s
Train Loss: 0.691 | Train Acc: 54.48%
Valid Loss: 0.681 | Valid Acc: 56.00%

```

모델 불러오기 & 테스트 평가

```

model.load_state_dict(torch.load('./chap06/data/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc*100:.2f}%')

```

- 목적: 학습이 끝난 모델(`VGG-model.pt`)을 불러와 테스트 데이터셋에서의 성능을 평가.
- `evaluate()` 함수: 검증 데이터와 동일한 구조로 **손실(loss)**과 **정확도(acc)**를 계산.
- 출력 예시:

```

Test Loss: 0.693 | Test Acc: 50.00%

```

예측 결과 얻기 (`get_predictions`)

```

def get_predictions(model, iterator):
    model.eval()
    images, labels, probs = [], [], []
    with torch.no_grad():
        for x, y in iterator:
            x = x.to(device)
            y_pred = model(x)
            y_prob = F.softmax(y_pred, dim=-1)
            top_pred = y_prob.argmax(1, keepdim=True)
            images.append(x.cpu())

```

```

labels.append(y.cpu())
probs.append(y_prob.cpu())
return torch.cat(images), torch.cat(labels), torch.cat(probs)

```

- `argmax(1)` → 각 행(이미지)에 대해 확률이 가장 높은 클래스 인덱스를 반환.
- `keepdim=True` → 차원을 유지하여 텐서 크기를 맞춤.
- `torch.cat()` → 여러 배치(batch)를 하나의 텐서로 연결.

올바르게 예측한 예시 추출

```

images, labels, probs = get_predictions(model, test_iterator)
pred_labels = torch.argmax(probs, 1)
corrects = torch.eq(labels, pred_labels)
correct_examples = []

for image, label, prob, correct in zip(images, labels, probs, corrects):
    if correct:
        correct_examples.append((image, label, prob))

```

- `torch.eq()` → 예측값(`pred_labels`)과 실제값(`labels`)이 같은지 비교.
- `zip()` → 여러 리스트를 묶어서 반복.
- `correct_examples` → 정확히 맞춘 예시만 저장.

정렬하기 (`sort` & `lambda`)

```

correct_examples.sort(reverse=True, key=lambda x: torch.max(x[2], dim=0).values)

```

- `reverse=True` → 높은 확률부터 내림차순 정렬.
- `key=lambda` → 각 예시의 최대 확률값(`x[2]`, 즉 `prob`)을 기준으로 정렬.
- `lambda` 함수는 익명 함수로, `lambda x: x+10` 처럼 간단히 정의 가능.

시각화를 위한 이미지 전처리

```

def normalize_image(image):
    image_min, image_max = image.min(), image.max()
    image.clamp_(min=image_min, max=image_max)
    image.add_(-image_min).div_(image_max - image_min + 1e-5)
    return image

```

- `clamp_()` : 이미지 값이 `[min, max]` 범위 내에 있도록 제한.
- `add_()` / `div_()` : `_`가 붙으면 in-place 연산 (메모리 재할당 없이 기존 텐서 수정).
- 결과적으로 이미지를 0~1 사이로 정규화하여 시각화할 수 있음.

올바르게 예측한 이미지 시각화

```
def plot_most_correct(correct, classes, n_images):
    rows = cols = int(np.sqrt(n_images))
    fig, ax = plt.subplots(rows, cols, figsize=(25,20))
    for i in range(rows*cols):
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0)
        true_class = classes[true_label]
        correct_prob, correct_label = torch.max(probs, dim=0)
        correct_class = classes[correct_label]
        image = normalize_image(image)
        ax.imshow(image.cpu().numpy())
        ax.set_title(f'True: {true_class} ({true_prob:.3f})\nPred: {correct_class} ({correct_prob:.3f})')
        ax.axis('off')
```

- `permute(1, 2, 0)` → (채널, 높이, 너비) → (높이, 너비, 채널)로 바꿔 matplotlib에서 올바르게 표시.
- `imshow()` → 이미지를 출력.
- `set_title()` → 정답 클래스와 예측 클래스, 확률 함께 표시.

결과 출력

```
classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)
```

→ 모델이 정확히 예측한 이미지 5개를 시각적으로 출력.

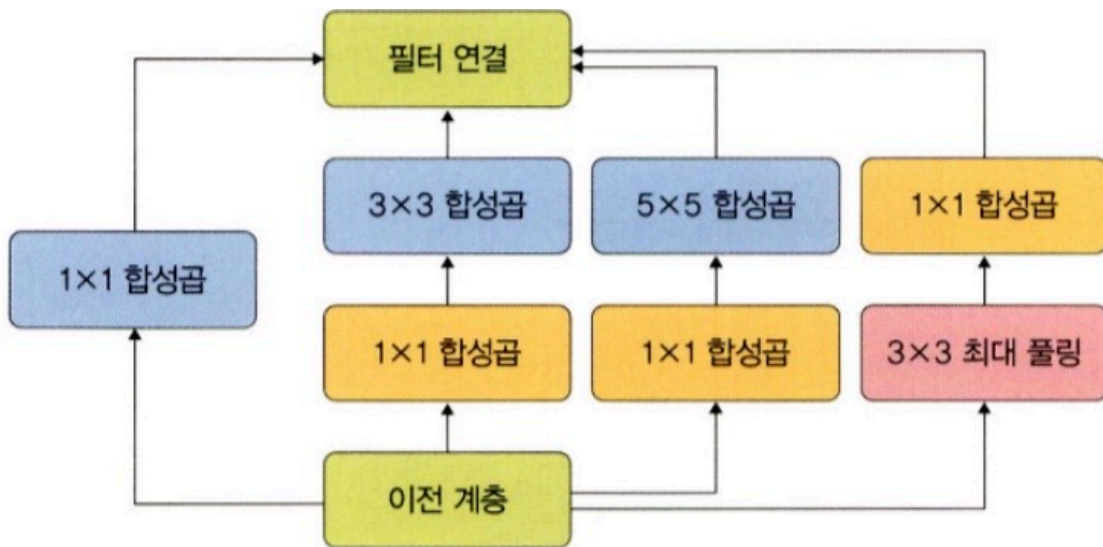
출력 결과에는 각 이미지의 True Label / Pred Label / 확률값이 함께 표시됨.

6.1.4 GoogLeNet (Inception Network)

개요

- GoogLeNet은 주어진 하드웨어 자원을 최대한 효율적으로 사용하면서 학습 성능을 극대화하기 위해 만들어진 깊고 넓은 신경망입니다.
- 2014년 Google이 발표한 Inception 구조를 기반으로 합니다.

인셉션(Inception) 모듈의 개념



- *인셉션 모듈(Inception Module)**은 GoogLeNet의 핵심 구조입니다.
- 다양한 크기의 필터(1×1, 3×3, 5×5)를 병렬로 동시에 적용해,
→ 여러 크기의 특징(feature)을 한 번에 추출할 수 있습니다.

구조

인셉션 모듈은 다음 네 가지 연산으로 구성됩니다:

1. 1×1 합성곱(Convolution)
2. 1×1 합성곱 + 3×3 합성곱
3. 1×1 합성곱 + 5×5 합성곱
4. 3×3 최대 풀링(Max Pooling) + 1×1 합성곱

Sparse Connectivity (희소 연결)

- 일반적인 CNN은 Dense(조밀한) 연결 구조로, 모든 노드가 연결되어 연산량이 많고 과적합(overfitting) 위험이 있습니다.
- GoogLeNet은 **희소 연결(sparse connectivity)**을 적용하여
→ 연산량을 줄이고,
→ 과적합 문제를 완화했습니다.

GoogLeNet의 목적

- 딥러닝에서 층이 깊고 넓어질수록 학습 효율과 인식력은 좋아지지만,
→ 연산량 증가, 과적합, 기울기 소멸(vanishing gradient) 등의 문제가 발생합니다.
- GoogLeNet은 Inception 구조를 통해 이 문제를 효율적으로 해결합니다.